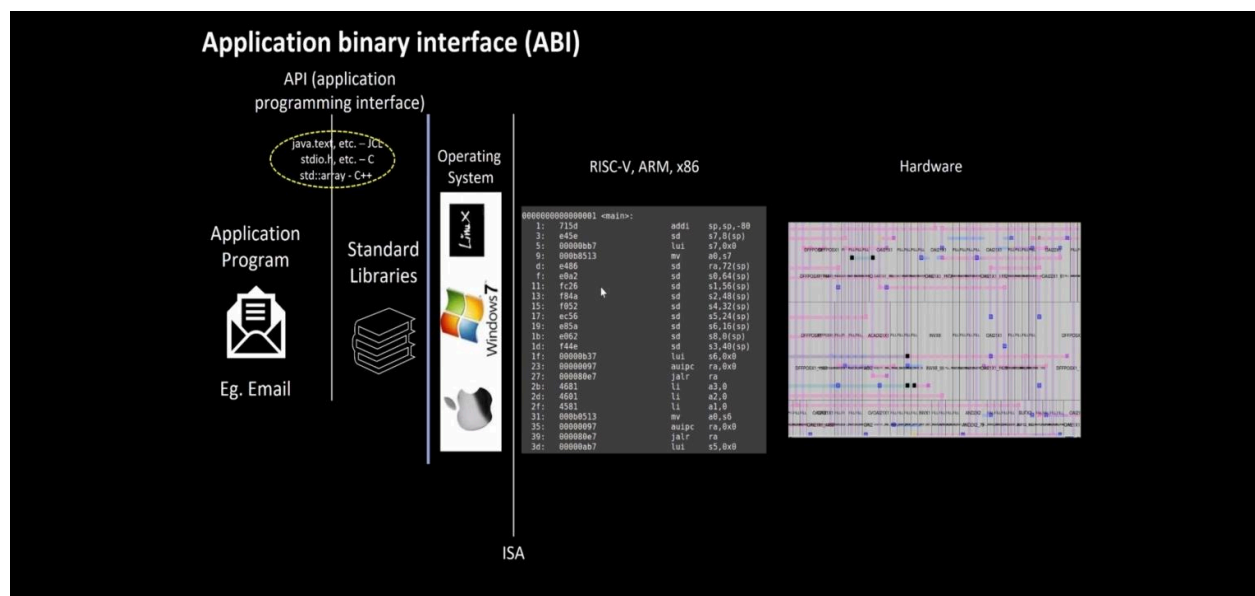


DAY 02

APPLICATION BINARY INTERFACE (ABI)

An Application Binary Interface (ABI) is the low-level connection between compiled software and the computer system. It defines how machine-level programs interact with the operating system, processor, and hardware. The concept is shown using assembly instructions, operating systems, standard libraries, and processor architectures such as x86, ARM, and RISC-V.



The explanation starts from a compiled program containing assembly instructions. These instructions follow ABI rules so the processor knows how to execute them correctly. The ABI specifies things like:

how function calls are made,

which CPU registers store arguments and return values,

how the stack and memory are organized,

how programs request operating system services through system calls.

The flow begins with an application program, such as an email app, which uses APIs from programming languages and libraries. Those libraries communicate with the operating system using system calls. The ABI defines the binary-level rules for this communication.

The operating system then interacts with the processor through the Instruction Set Architecture (ISA), such as RISC-V, ARM, or x86. Beneath that is the actual hardware implementation. Different processors may be designed for high performance or low power, but if they follow the same ABI and ISA rules, the same compiled program can still run correctly.

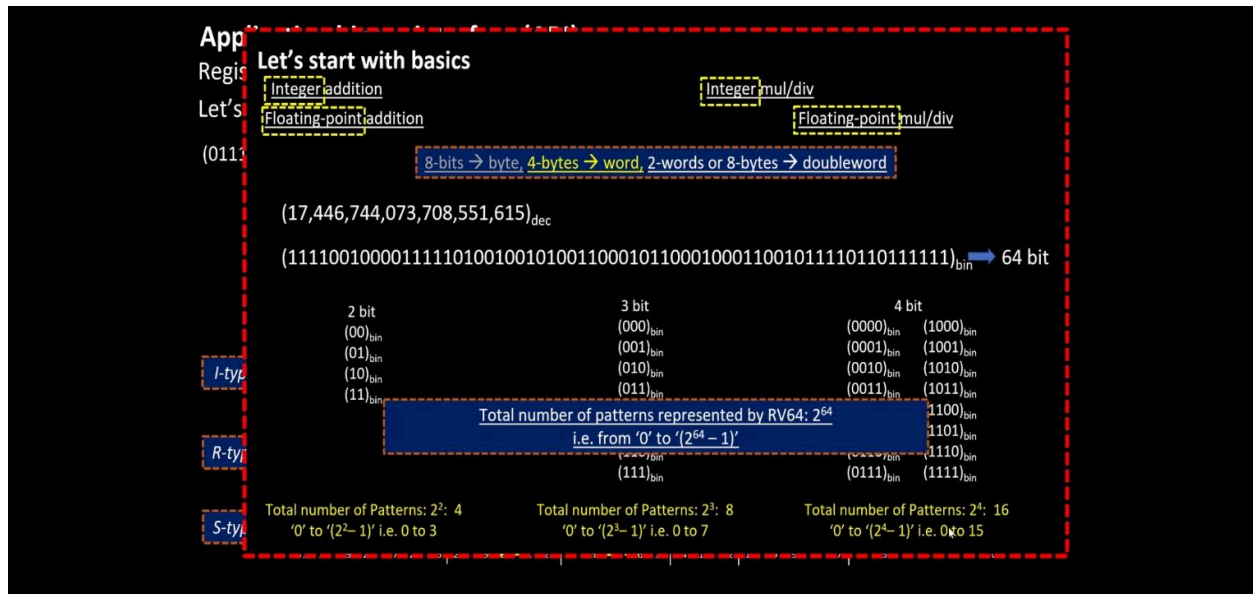
In simple terms:

API helps programmers write software.

ABI helps compiled software run correctly on a system.

ISA defines the CPU instructions understood by the processor.

The main idea is that the ABI acts as a stable bridge between software and hardware, ensuring compatibility at the machine-code level.



A computer works using binary numbers, which are made of only 0s and 1s called bits. More bits can represent more values. For example, 2 bits can store 4 values, 3 bits can store 8 values, and 4 bits can store 16 values. In computers, 8 bits make 1 byte, 4 bytes make 1 word, and 8 bytes make 1 doubleword. The processor performs two main types of calculations: integer operations for whole numbers and floating-point operations for decimal numbers.

The processor also contains small and very fast storage locations called registers, which are used during calculations and program execution. In RISC-V, there are 32 registers named x0 to x31, and each has a specific purpose. For example, x0 always

stores 0, x1 stores the return address after a function call, x2 acts as the stack pointer, and some registers are used for temporary values or passing function arguments. Functions also follow rules about saving and restoring register values, known as caller-saved and callee-saved registers.

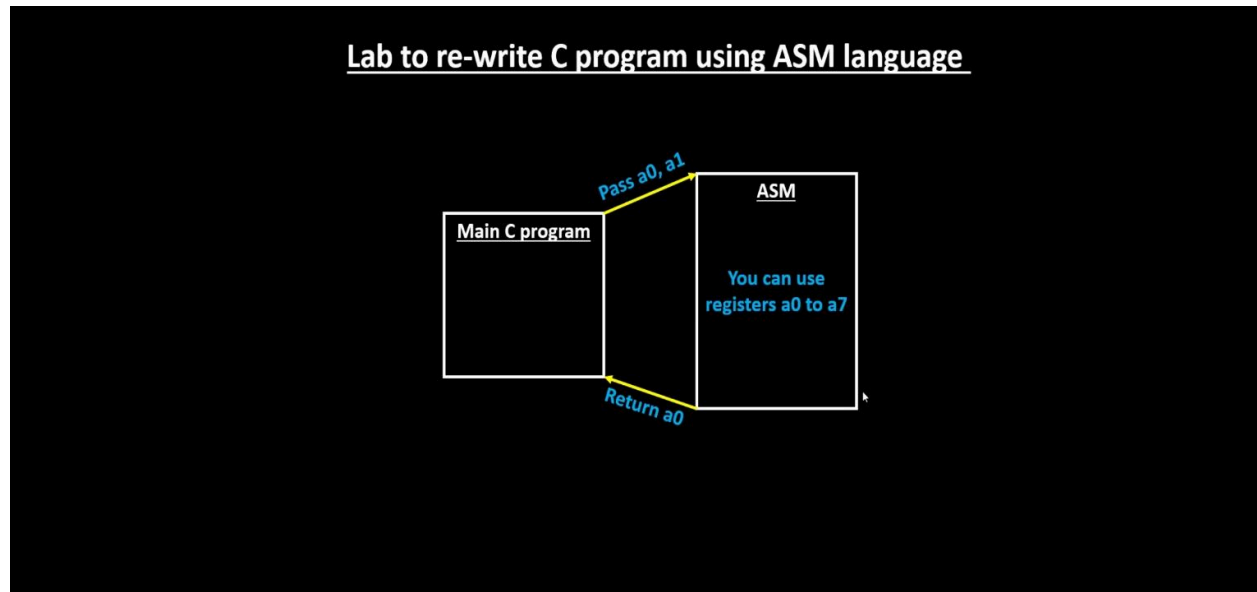
Application binary interface (ABI)					
Register	ABI name	Usage	Saver		
x0	zero	Hard-wired zero	-		x0
x1	ra	Return address	Caller		x1
x2	sp	Stack pointer	Callee		x2
x3	gp	Global pointer	-		x3
x4	tp	Thread pointer	-		x4
x5-7	t0-2	Temporaries	Caller		x5
x8	s0/fp	Saved register/frame pointer	Callee		x6
x9	s1	Saved register	Callee		x7
x10-11	a0-1	Function arguments/return values	Caller		
x12-17	a2-7	Function arguments	Caller		
x18-27	s2-11	Saved registers	Callee		
x28-31	t3-6	Temporaries	Callee		



Registers

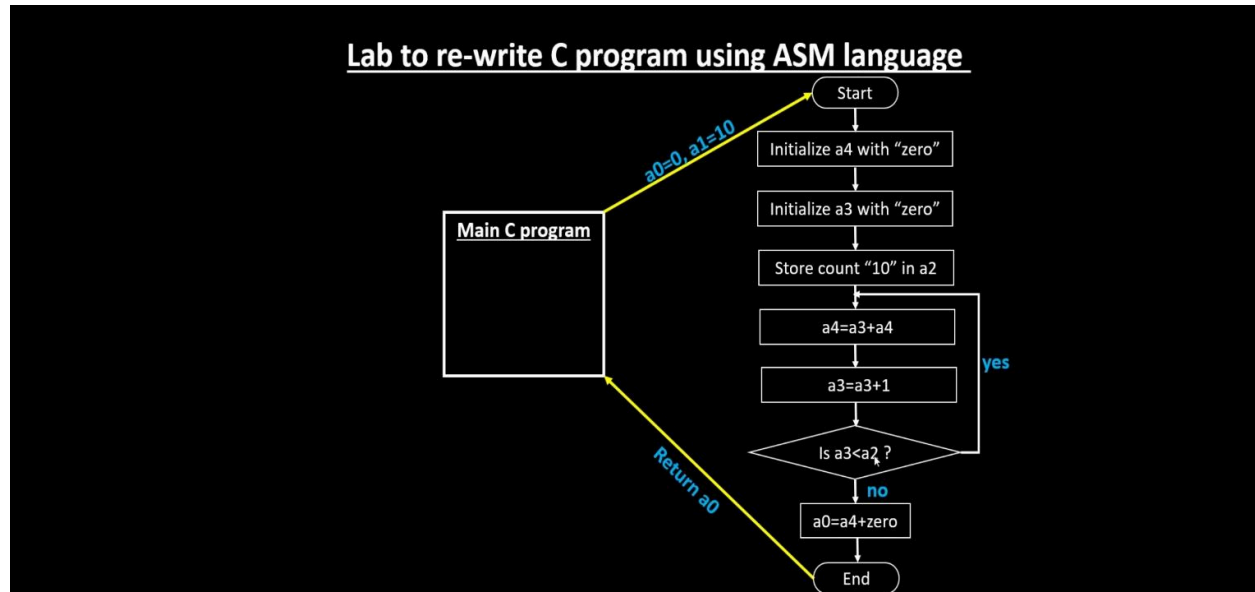
- x24
- x25
- x26
- x27
- x28
- x29
- x30
- x31

Assembly language (ASM)



The flowchart explains a simple program that is first written in C language and then rewritten in Assembly (ASM) language. The program starts by setting two registers, **a3** and **a4**, to zero. Another register, **a2**, is loaded with the value 10, which acts as the loop limit or counter. Inside the loop, the value of **a3** is added to **a4**, so **a4** keeps storing the running total. After that, **a3** is increased by 1. The program then checks whether **a3** is still less than **a2**. If the condition is true, the loop repeats again. If the condition becomes false, the final value stored in **a4** is moved to **a0**, which is used as the return value, and the program ends. In simple terms, the program

calculates the sum of numbers from 0 to 9 using Assembly language instructions.



EXAMPLE LAB:

1. Open the terminal in Ubuntu/Linux.
2. Type the following command and press Enter:

leafpad 1to9_custom.c &

3. The following C program will appear in Leafpad:

```
#include <stdio.h>
```

```
extern int load(int x, int y);
```

```
int main() {  
    int result = 0;  
    int count = 9;  
    result = load(0x0, count+1);  
    printf("Sum of number from 1 to %d is %d\n",  
count, result);  
}
```

4. Again open the terminal and type:

leafpad load.S &

5. The following Assembly program will appear in
Leafpad:

```
.section .text  
.global load  
.type load, @function
```

load:


```
    add    a4, a0, zero    // Initialize sum register a4  
with 0
```

```
    add    a2, a0, a1      // Store count value in  
register a2
```

```
    add    a3, a0, zero    // Initialize loop counter a3  
with 0
```

loop:

```
    add    a4, a3, a4      // Add counter value to sum
```

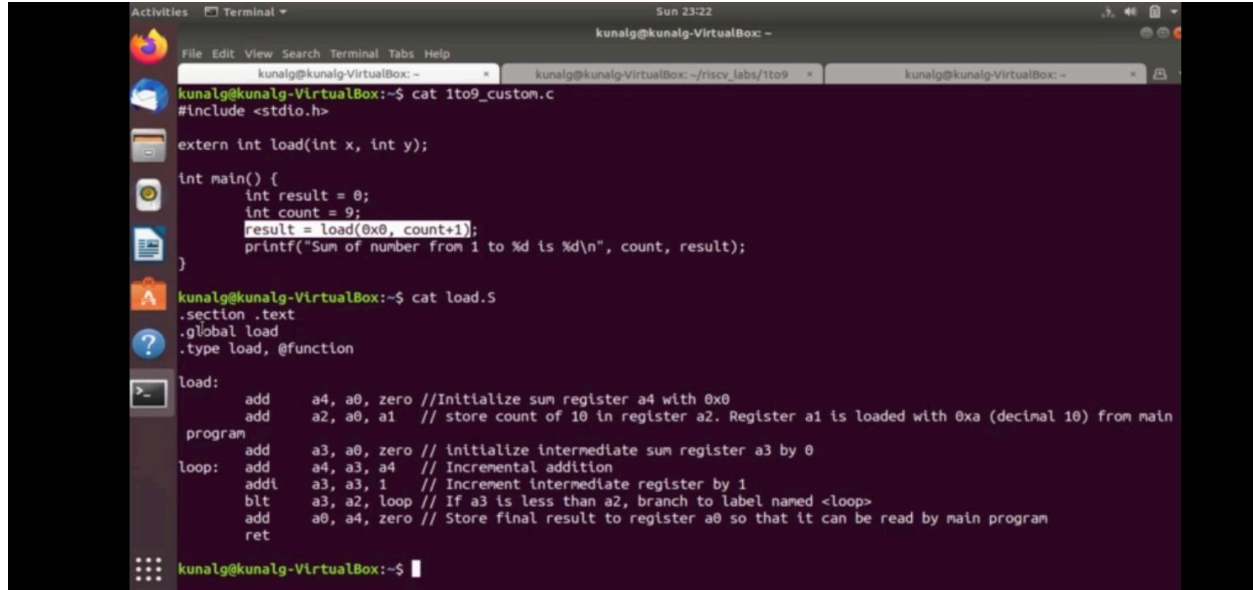
```
    addi   a3, a3, 1       // Increment counter by 1
```

```
    blt    a3, a2, loop    // Repeat loop if a3 < a2
```

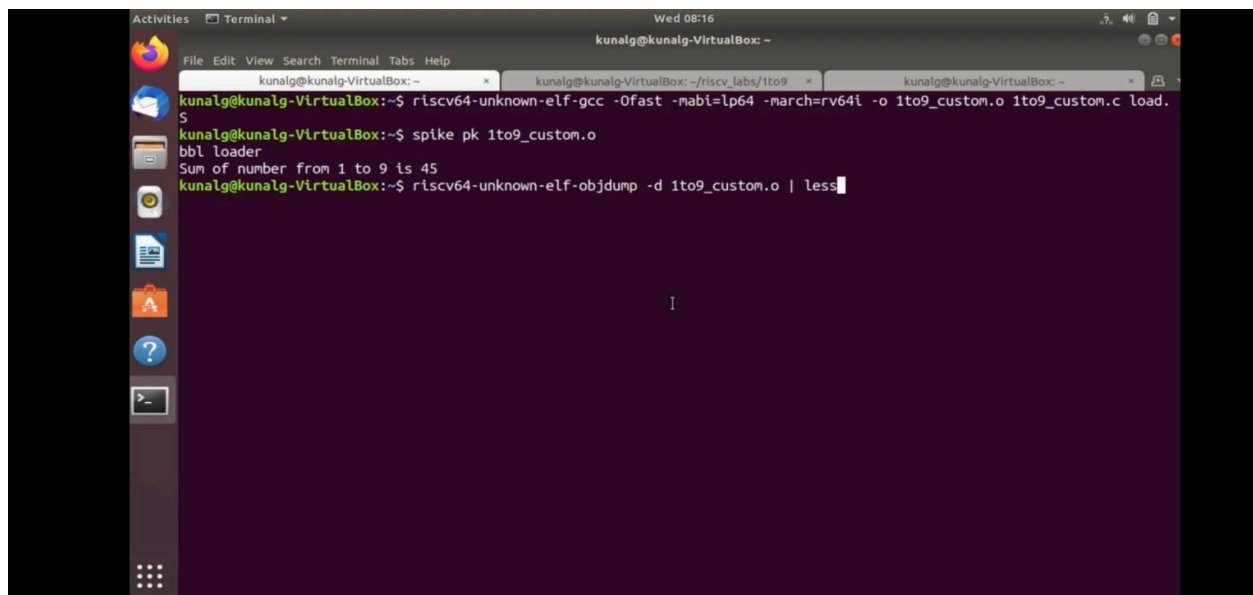
```
    add    a0, a4, zero    // Move final result to a0
```

```
    ret                    // Return result
```

THEN CONTINUE:



```
kunalg@kunalg-VirtualBox: ~  
File Edit View Search Terminal Tabs Help  
kunalg@kunalg-VirtualBox:~$ cat 1to9_custom.c  
#include <stdio.h>  
  
extern int load(int x, int y);  
  
int main() {  
    int result = 0;  
    int count = 9;  
    result = load(0x0, count+1);  
    printf("Sum of number from 1 to %d is %d\n", count, result);  
}  
  
kunalg@kunalg-VirtualBox:~$ cat load.S  
.section .text  
.global load  
.type load, @function  
  
load:  
    add    a4, a0, zero //Initialize sum register a4 with 0x0  
    add    a2, a0, a1   // store count of 10 in register a2. Register a1 is loaded with 0xa (decimal 10) from main  
program  
    add    a3, a0, zero // initialize intermediate sum register a3 by 0  
loop:  
    add    a4, a3, a4   // Incremental addition  
    addi   a3, a3, 1    // Increment intermediate register by 1  
    blt    a3, a2, loop // If a3 is less than a2, branch to label named <loop>  
    add    a0, a4, zero // Store final result to register a0 so that it can be read by main program  
    ret  
  
kunalg@kunalg-VirtualBox:~$
```



```
kunalg@kunalg-VirtualBox: ~  
File Edit View Search Terminal Tabs Help  
kunalg@kunalg-VirtualBox:~$ riscv64-unknown-elf-gcc -Ofast -mabi=lp64 -march=rv64i -o 1to9_custom.o 1to9_custom.c load.S  
kunalg@kunalg-VirtualBox:~$ spike pk 1to9_custom.o  
bbl loader  
Sum of number from 1 to 9 is 45  
kunalg@kunalg-VirtualBox:~$ riscv64-unknown-elf-objdump -d 1to9_custom.o | less
```

1. Use the following command to display the C program file contents:

```
cat 1to9_custom.c
```

2. The C program will be displayed on the terminal screen.

3. Use the following command to display the Assembly program file contents:

```
cat load.S
```

4. The Assembly program with comments will appear on the terminal screen.

5. Type the following command to clean the terminal screen:

```
clear
```

6. Use the following command to show the files in the folder:

```
ls
```

7. Compile the C and Assembly programs using the following command:

```
riscv64-unknown-elf-gcc -Ofast -mabi=lp64  
-march=rv64i -o 1to9_custom.o 1to9_custom.c load.S
```

8. Execute the generated object file using Spike simulator:

```
spike pk 1to9_custom.o
```

9. The output will be displayed as:

Sum of number from 1 to 9 is 45

10. Use the following command to view the disassembled object code:

```
riscv64-unknown-elf-objdump -d 1to9_custom.o | less
```

LAB VERIFICATION:

Step 1: Open Home Directory

```
cd
```

Step 2: Clone the GitHub Repository

```
git clone
```

```
https://github.com/kunalg123/riscv\_workshop\_collaterals.git
```

Step 3: Enter into Repository Folder

```
cd riscv_workshop_collaterals
```

Step 4: List the Files

```
ls -ltr
```

Step 5: Move to Labs Directory

```
cd labs
```

Step 6: List Files in Labs Folder

```
ls -ltr
```

Step 7: Open PicoRV32 Verilog File

```
vim picorv32.v
```

Step 8: View PicoRV32 File

```
less picorv32.v
```

Step 9: Open Testbench File

```
vim testbench.v
```

Step 10: Open Shell Script

```
vim rv32im.sh
```

Step 11: Give Execute Permission

```
chmod 777 rv32im.sh
```

Step 12: Run the Shell Script

```
./rv32im.sh
```

Step 13: Open Custom C File

```
vim 1to9_custom.c
```

Step 14: Run the Script Again

```
./rv32im.sh
```

Step 15: List Generated Files

```
ls -ltr
```

Step 16: Open Firmware HEX File

```
vim firmware.hex
```

Step 17: Open Firmware32 HEX File

```
vim firmware32.hex
```

Step 18: Open Testbench File Again

```
vim testbench.v
```